

Module 5 Week A — Core Skills Drill: Classification & Evaluation Basics

This drill is due the evening of the concept session (Sunday). It moves you from “I read about train/test splits and metrics” to “I can implement splitting and calculate metrics from code.” Complete all three tasks, open a PR, and submit your PR URL.

Setup

1. **Accept the GitHub Classroom assignment:** [Accept the Drill 5A assignment](#)
2. **Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```
3. **Install dependencies:**

```
pip install -r requirements.txt
```

This installs **scikit-learn** for the first time. If you encounter version conflicts, create a fresh virtual environment first (`python -m venv .venv && source .venv/bin/activate`), then install.
4. **Create a working branch:**

```
git checkout -b drill-5a-classification-basics
```

All your work goes in `drill.py`. The starter file contains function signatures and docstrings – implement each function as described below.

Task 1: Split a Dataset Correctly

- You are given a CSV loaded into a DataFrame with a binary target column called `churned`.
- Your task:**
- Use `train_test_split` to create an 80% training set and 20% test set.
 - Set `random_state=42` for reproducibility.
 - Set `stratify` on the target column to preserve class distribution.
 - After splitting, verify that the split sizes are correct and that the class distribution is preserved (churn rate in train and test should be within 2 percentage points of the original).
- What your function returns:** A tuple of `(X_train, X_test, y_train, y_test)`.

Task 2: Calculate Classification Metrics

- Your function receives two arrays as arguments: `y_true` (the actual labels) and `y_pred` (the model’s predicted labels). The autograder generates these by training a model on your Task 1 split and passing the results to your function.
- Your task:**
- Compute accuracy, precision, recall, and F1-score using scikit-learn metric functions.
 - Store the results in a dictionary with these exact keys: `"accuracy"`, `"precision"`, `"recall"`, `"f1"`.
 - All values should be floats.
- What your function returns:** A dictionary with four keys and their corresponding metric values.

Task 3: Run Cross-Validation

- Your task:**
- Train a `LogisticRegression` model with `random_state=42`, `max_iter=1000`, and `class_weight="balanced"`. The `class_weight="balanced"` parameter upweights the minority class during training. Without it, the model would predict only the majority class and produce zero precision and recall; with it, the model catches more minority-class examples at the cost of overall accuracy. This is the right trade-off when the minority class is what you care about detecting — Section 5 of the Week A reading explains why accuracy alone can mislead on imbalanced datasets.
 - Run 5-fold stratified cross-validation on the provided training data using `cross_val_score` with `scoring="accuracy"`.
 - Compute the mean and standard deviation of the CV scores.
- What your function returns:** A dictionary with keys `"scores"` (array of fold scores), `"mean"` (float), and `"std"` (float).

Submission

1. Commit and push your completed `drill.py`:

```
git add drill.py
git commit -m "Complete drill 5A tasks"
git push origin drill-5a-classification-basics
```
2. Open a pull request from `drill-5a-classification-basics` to `main`.
3. Confirm the autograder passes (green check on your PR).
4. Paste your PR URL into TalentLMS -> Module 5 Week A -> Core Skills Drill 5A to submit this assignment.